

The Definitive Guide to PRD Writing

Blueprinting Software for AI Agents

DOTE 3010

Abstract

This document is an extensive guide designed for business students with zero coding background. It explains not just *how* to fill out a Product Requirements Document (PRD), but *why* each section is critical when managing an AI Developer (Agent). By mastering this document, you learn to "program in English."

Contents

Introduction: The CEO and The Intern	2
Part 1: The "Vibe" (Philosophy of Constraints)	2
Defining the Tech Stack	2
Part 2: Anatomy of a Perfect PRD	2
1. The One-Liner	3
2. User Roles	3
3. User Stories	3
4. Data Structure	3
5. Non-Functional Requirements	4
Part 3: The "Golden" Template	5
Part 4: Reviewing Your PRD	6
Conclusion	6

Introduction: The CEO and The Intern

Imagine you are the CEO of a company. You have just hired a brilliant but incredibly literal intern named "AI".

- This intern reads 10,000 words per minute.
- This intern knows every programming language in history.
- **However**, this intern has zero "common sense" about your business.
- If you tell them to "clean the office," they might throw away your desk because it had dust on it.

Writing a PRD for Vibe Coding (AI-assisted programming) is the art of writing instructions so precise that even a literal-minded genius cannot misunderstand them. In traditional software development, a Product Manager talks to a Human Engineer, and they clarify things over coffee. With AI, **the PRD is the only truth**. If it's not in the PRD, it doesn't exist in the software.

Part 1: The "Vibe" (Philosophy of Constraints)

The biggest mistake business students make is being too "open-minded." When you say "Use whatever technology is best," the AI is paralyzed by choice or chooses something overly complex (like a microservices architecture for a lemonade stand app).

The Golden Rule

Constraint is Freedom. By restricting the AI's choices, you force it to focus on the logic rather than the architecture.

Defining the Tech Stack (Even if you don't understand it)

You don't need to know how to write Python, but you must mandate it. Why? Because if you let the AI choose esoteric languages (like Haskell or Rust), you won't be able to debug it later.

Copy-Paste Recommendations for DOTE Projects:

- **Backend Language:** Python (Universally understood, great for data/AI).
- **Backend Framework:** FastAPI (Modern, fast, AI loves writing it).
- **Database:** SQLite (for simple prototypes) or PostgreSQL (for serious apps). Avoid complex setups like Oracle.
- **Frontend:** HTML + TailwindCSS + Vanilla JavaScript.
 - *Why?* Modern frameworks like React require complex "Build Steps" (Node.js, Webpack) which confuse beginners. A single HTML file is easier to manage.

Part 2: Anatomy of a Perfect PRD

A PRD is broken down into specific sections. Do not skip any.

1. The One-Liner (Scope Definition)

Start with a single sentence that describes what the software DOES, not what it "is".

- **Bad:** "A platform for connecting students." (Too vague. Is it Tinder? LinkedIn? WhatsApp?)
- **Good:** "A web-based message board where CUHK students can post second-hand textbooks for sale and buyers can comment to reserve them."

2. User Roles (The Actors)

Who is using the system? Different users need different data visibility.

- **Guest:** Can view books. Cannot post.
- **Student Seller:** Can post books. Can delete *their own* books.
- **Admin:** Can delete *any* book (for moderation).

Why this matters: The AI uses this to write "Permissions Logic" (e.g., `if user.role == 'admin': allow_delete()`).

3. User Stories (The Functional Requirements)

This is the heart of the document. Use the standard format:

"As a [Role], I want to [Action], so that [Benefit]."

Detailed Examples:

1. **The Happy Path:** "As a Seller, I want to upload a photo of my book, so buyers can see its condition."
2. **The Edge Case:** "As a System, I want to reject images larger than 5MB, so that the server storage doesn't fill up."
3. **The Negative Path:** "As a Buyer, I want to see an error message if I try to buy a book that has already been sold."

Tip: Be Specific about "Where"

Don't just say "I want to log in." Say "I want a Login Modal to pop up on the homepage."

4. Data Structure (The "Memory" Schema)

This is crucial for Business Students. Think of your software as a set of Excel Sheets linked together. You must define the columns. If you don't list a column, the AI won't create it, and the data won't be saved.

Example: The "Textbook" Table

- **id:** Unique number (1, 2, 3...)
- **title:** Text string (e.g., "Economics 101")
- **price:** Number (Decimal, 2 decimal places)
- **condition:** Selection (New, Good, Poor) – *Tell the AI these are the only 3 options!*
- **seller_id:** Link to the User Table.
- **is_sold:** Boolean (True/False). Default is False.

5. Non-Functional Requirements (The Quality Control)

These are invisible features.

- **Performance:** "The search result must appear within 2 seconds."
- **Mobile:** "The layout must be vertical on mobile screens."
- **Security:** "Passwords must be hashed (encrypted), never stored as plain text."

Part 3: The "Golden" Template

(Copy this section into your own document and fill in the blanks. Delete the instructional text in brackets.)

Project Name: [Insert Name]

Version: 1.0

Author: DOTE 3010 Student

1. Product Overview

Problem: [Describe the pain point, e.g., "Students throw away notes that juniors could use."]

Solution: [Describe the app, e.g., "A digital marketplace for notes."]

Platform: Web Browser (Chrome/Safari).

2. Tech Stack Constraints

- Backend: Python (FastAPI)
- Database: SQLite (Local file)
- Frontend: HTML5 + TailwindCSS (Single file if possible)
- Auth: Simple Cookie/Session based.

3. Data Schema (The Database)

Table: Users

- id, email (unique), password_hash, created_at

Table: Products

- id, name, description, price, owner_id (Foreign Key)

4. Core Features (MVP Scope)

Feature 1: User Authentication

- User can register with Email/Password.
- System checks if Email already exists.
- User can login/logout.

Feature 2: Product Management

- Logged-in users can create a Product.
- Required fields: Name, Price. Optional: Description.
- Users can view a list of all Products on the home page.

5. UI Guidelines

- Use a dark theme (Slate-900 background).
 - Use blue buttons for primary actions ("Create", "Buy").
 - Use red buttons for destructive actions ("Delete").
-

Part 4: Reviewing Your PRD

Before you give this document to the AI, perform the "Am I Ambiguous?" test.

1. Did I use words like "User-friendly" or "Modern"? (Delete them. Define "Big buttons" or "Minimalist" instead).
2. Did I explain what happens when things go wrong? (e.g., "What if the user uploads a virus?" -> "Restrict file types to .jpg only").
3. Did I define the relationships? (e.g., "Can a user have multiple products?" -> Yes/No).

Conclusion

A good PRD is boring. It should read like a legal contract or an instruction manual for IKEA furniture. The more boring and specific it is, the better code your AI agent will generate. Creativity belongs in the *idea*, not in the *instructions*.